



# Semaine 4 : Les fonctions

**Objectif** : Créer des blocs de code réutilisables

**Durée** : 2 heures

**Lien Scratch** : Les blocs personnalisés 



## Ce que tu vas apprendre

1. Créer une fonction
2. Appeler une fonction
3. Les paramètres
4. Retourner une valeur avec `return`
5. Organiser son code

## 1 Qu'est-ce qu'une fonction ?

Une fonction, c'est un **bloc de code réutilisable** avec un nom.  
C'est comme les **blocs personnalisés** dans Scratch !

### Scratch vs JavaScript

 Définir saluer

Dire "Bonjour !"

```
function saluer() {  
    console.log("Bonjour !");  
}
```

### Pourquoi utiliser des fonctions ?

1. **Éviter de répéter** le même code
2. **Organiser** son programme
3. **Nommer** des actions (plus lisible)

## 2 Créer et appeler une fonction

---

### Syntaxe de base

```
// 1. DÉCLARER la fonction (créer le bloc)
function direBonjour() {
    console.log("Bonjour !");
    console.log("Comment ça va ?");
}

// 2. APPELER la fonction (utiliser le bloc)
direBonjour(); // Affiche les deux messages
direBonjour(); // On peut l'appeler plusieurs fois !
```

### Exemple : Afficher une ligne de décoration

```
function afficherLigne() {
    console.log("=====");
}

afficherLigne();
console.log("    Bienvenue dans le jeu !");
afficherLigne();
```

Résultat :

```
=====
    Bienvenue dans le jeu !
=====
```



**Exercice :** `exercice_1_fonctions_simples.js`

---

### 3 Les paramètres : donner des informations à la fonction

---

Un paramètre permet de passer des valeurs à la fonction.

#### Scratch vs JavaScript

■ Définir saluer (nom)

Dire "Bonjour " + nom

```
function saluer(nom) {  
    console.log(`Bonjour ${nom}`)  
}
```

#### Exemple avec un paramètre

```
function saluer(nom) {  
    console.log(`Bonjour ${nom} !`);  
}  
  
saluer("Steve");    // Affiche : Bonjour Steve !  
saluer("Alex");     // Affiche : Bonjour Alex !  
saluer("Creeper");  // Affiche : Bonjour Creeper !
```

#### Exemple avec plusieurs paramètres

```
function afficherCombat(joueur, monstre) {  
    console.log(`⚔️ ${joueur} combat ${monstre} !`);  
}  
  
afficherCombat("Steve", "Zombie");    // Steve combat Zombie !  
afficherCombat("Alex", "Squelette");   // Alex combat Squelette !
```

## Paramètres avec valeur par défaut

```
function saluer(nom = "Aventurier") {  
    console.log(`Bonjour ${nom} !`);  
}  
  
saluer("Steve"); // Bonjour Steve !  
saluer();        // Bonjour Aventurier ! (valeur par défaut)
```

 Exercice : `exercice_2_parametres.js`

## 4 Retourner une valeur avec `return`

Une fonction peut **calculer** quelque chose et **renvoyer** le résultat.

### Sans return (affiche seulement)

```
function calculerDouble(nombre) {  
    console.log(nombre * 2); // Affiche, mais ne renvoie rien  
}  
  
calculerDouble(5); // Affiche 10  
let resultat = calculerDouble(5); // resultat = undefined 😞
```

### Avec return (renvoie le résultat)

```
function calculerDouble(nombre) {  
    return nombre * 2; // Renvoie la valeur  
}  
  
let resultat = calculerDouble(5);  
console.log(resultat); // Affiche 10  
console.log(calculerDouble(7)); // Affiche 14
```

## Exemples pratiques

```
// Calculer l'aire d'un rectangle
function aire(largeur, hauteur) {
    return largeur * hauteur;
}

let surface = aire(10, 5);
console.log(`Aire = ${surface}`); // Aire = 50

// Vérifier si un nombre est pair
function estPair(nombre) {
    return nombre % 2 === 0;
}

console.log(estPair(4)); // true
console.log(estPair(7)); // false

// Utiliser dans un if
if (estPair(10)) {
    console.log("10 est pair !");
}
```



Exercice : `exercice_3_return.js`

## 5 Fonctions qui appellent d'autres fonctions

On peut utiliser une fonction dans une autre !

```
function carre(n) {
    return n * n;
}

function cube(n) {
    return n * carre(n); // Utilise carre() !
}

console.log(carre(3)); // 9
console.log(cube(3)); // 27
```

### Exemple : Système de combat

```
function calculerDegats(attaque, defense) {
    let degats = attaque - defense;
    if (degats < 0) degats = 0; // Minimum 0
    return degats;
}

function attaquer(nomAttaquant, attaque, nomDefenseur, defense, pv) {
    let degats = calculerDegats(attaque, defense);
    pv = pv - degats;
    console.log(`${nomAttaquant} attaque ${nomDefenseur} !`);
    console.log(`💣 ${degats} dégâts ! PV restants : ${pv}`);
    return pv;
}

let pvZombie = 20;
pvZombie = attaquer("Steve", 15, "Zombie", 5, pvZombie);
// Steve attaque Zombie !
// 💣 10 dégâts ! PV restants : 10
```

 **Exercice :** exercice\_4\_fonctions\_avancees.js

## 6 Organiser son code avec les fonctions

---

### Mauvais exemple (tout mélangé)

```
// Difficile à lire et maintenir...
let pv = 20;
console.log("Bienvenue !");
let choix = "1";
if (choix === "1") {
    pv -= 5;
    console.log("Tu perds 5 PV");
}
console.log(`PV: ${pv}`);
```

### Bon exemple (avec fonctions)

```
function afficherBienvenue() {
    console.log("Bienvenue !");
}

function subirDegats(pv, degats) {
    pv -= degats;
    console.log(`Tu perds ${degats} PV`);
    return pv;
}

function afficherPV(pv) {
    console.log(`PV: ${pv}`);
}

// Programme principal clair et lisible
let pv = 20;
afficherBienvenue();
pv = subirDegats(pv, 5);
afficherPV(pv);
```

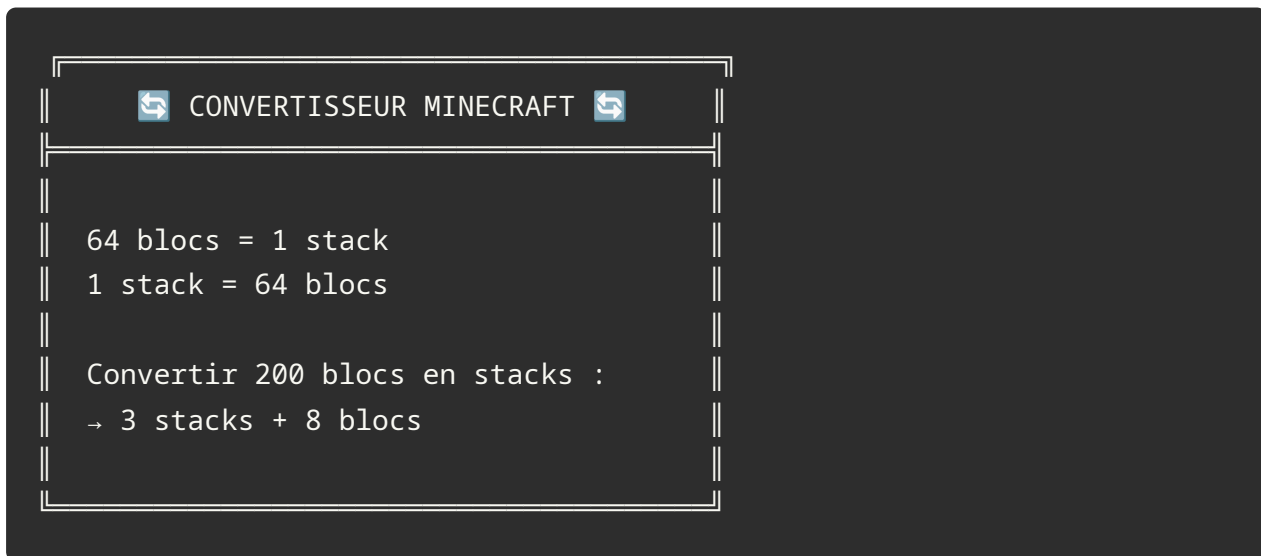
 **Exercice :** `exercice_5_organisation.js`

## Résumé

| Concept           | Syntaxe                                |
|-------------------|----------------------------------------|
| Déclarer          | <code>function nom() { }</code>        |
| Appeler           | <code>nom();</code>                    |
| Paramètre         | <code>function nom(param) { }</code>   |
| Plusieurs params  | <code>function nom(a, b, c) { }</code> |
| Valeur par défaut | <code>function nom(a = 10) { }</code>  |
| Retourner         | <code>return valeur;</code>            |

## Mini-projet : Convertisseur d'unités

À la fin de cette séance, tu sauras créer :





## Prochaine étape

---

1. Termine par le défi bonus : `defi_convertisseur.js`
2. Ensuite : le **Challenge final du Module 1** ! 🎮